



Addis Ababa Science and Technology University

Department of Software Engineering

Meal Management System

Database Systems Group Project Report

Prepared By:

Group 6

Instructor:

Mr. Yaynshet Medhn

Submission Date: May 22, 2026

Table of Contents

1. Chapter One – Introduction
2. Chapter Two – Database Design
3. Chapter Three – Implementation
4. Project Timeline
5. Conclusion
6. References
7. Additional Screenshots/images

CHAPTER ONE

INTRODUCTION

1.1 Background of the Organization

University meal management services are responsible for providing daily meals to students during different meal sessions such as breakfast, lunch, and dinner. These services involve managing student meal access, meal schedules, cafeteria operations, and transaction records.

In many institutions, meal-related information is handled manually or through unorganized systems. This can create difficulties in tracking student meal usage, recording transactions accurately, and generating reports efficiently.

A database management system can help organize and manage this information in a structured manner. The proposed Meal Management System is designed to store and manage student information, meal sessions, menu details, and meal transaction records efficiently.

The system will improve data organization, support accurate transaction recording, and assist the university in managing cafeteria operations effectively.

1.2 Problem Statement

University meal service operations involve several activities such as recording student meal access, managing meal sessions, organizing menu information, and tracking meal transactions.

When these operations are managed manually or without a properly designed database system, several challenges may arise.

These challenges include:

- Difficulty in storing and retrieving meal transaction records efficiently
- Poor organization of student meal information
- Duplicate meal access attempts
- Inaccurate or inconsistent data recording
- Difficulty in generating reports related to meal usage
- Inefficient management of cafeteria operations

Therefore, there is a need to design and implement a database system that supports efficient data management, organized storage, accurate transaction recording, and report generation for university meal management services.

1.3 General Objective

To design and implement a database system for managing university meal services efficiently.

1.4 Specific Objectives

- To analyze the requirements of a university meal management system
- To design a relational and non-relational database structure
- To manage student information efficiently
- To record and track meal transactions
- To manage meal sessions and menu information
- To reduce redundancy and improve data consistency
- To generate reports related to meal transactions and meal usage patterns
- To implement the database using MySQL and MongoDB

1.5 Scope of the Project

This project focuses on designing and implementing a database system for university meal management services.

Included in the Scope

- Student information management
- Meal session management
- Menu management
- Meal transaction recording
- Database implementation using MySQL and MongoDB
- Storage and retrieval of meal-related information
- Report generation based on stored data

Out of Scope

- Mobile application development
- Online payment integration
- Biometric authentication systems
- Real-time cafeteria monitoring systems

1.6 Significance of the Project

For the University

- Provides organized storage of meal-related information
- Improves management of meal transactions
- Supports accurate report generation
- Reduces redundancy and data inconsistency
- Improves operational efficiency

For Students

- Supports accurate meal transaction recording
- Reduces meal access errors
- Improves organization of cafeteria services

1.7 Methodology and Tools

Methodology

The following methodology will be followed during the development of the project:

1. Requirement analysis
2. Database design
3. ER diagram creation
4. Normalization
5. Database implementation
6. Testing and validation
7. Documentation

Tools and Technologies

- MySQL
- MongoDB
- MySQL Workbench
- MongoDB Compass
- Draw.io / Lucidchart
- GitHub
- Microsoft Word

1.8 System Requirements

1.8.1 Functional Requirements

The system shall:

- Store and manage student information
- Store and manage meal session information
- Record meal transactions
- Store menu information
- Track meal access timestamps
- Maintain transaction history
- Allow retrieval of stored records
- Generate meal transaction reports
- Support database queries and updates

1.8.2 Non-Functional Requirements

The system shall:

- Ensure data accuracy and consistency
- Provide efficient data storage and retrieval
- Support scalability for increasing student records
- Maintain system reliability and availability
- Ensure data security and controlled access
- Be maintainable and easy to update
- Support efficient database performance

CHAPTER TWO

DATABASE DESIGN

2.1 Requirement Analysis

The Meal Management System requires proper management of students, meal sessions, meal transactions, and menu information.

The system should allow the university cafeteria to manage meal-related operations efficiently and maintain organized transaction records.

The primary users of the system are:

- Cafeteria administrators
- Meal service staff
- University students

The system must support data storage, retrieval, updating, and report generation.

2.2 Entity Identification

The following entities are identified in the system:

STUDENT

Stores student-related information.

MEAL_TRANSACTION

Stores records of meal transactions.

MEAL_SESSION

Stores meal session information such as breakfast, lunch, and dinner.

MENU

Stores menu information for meal sessions.

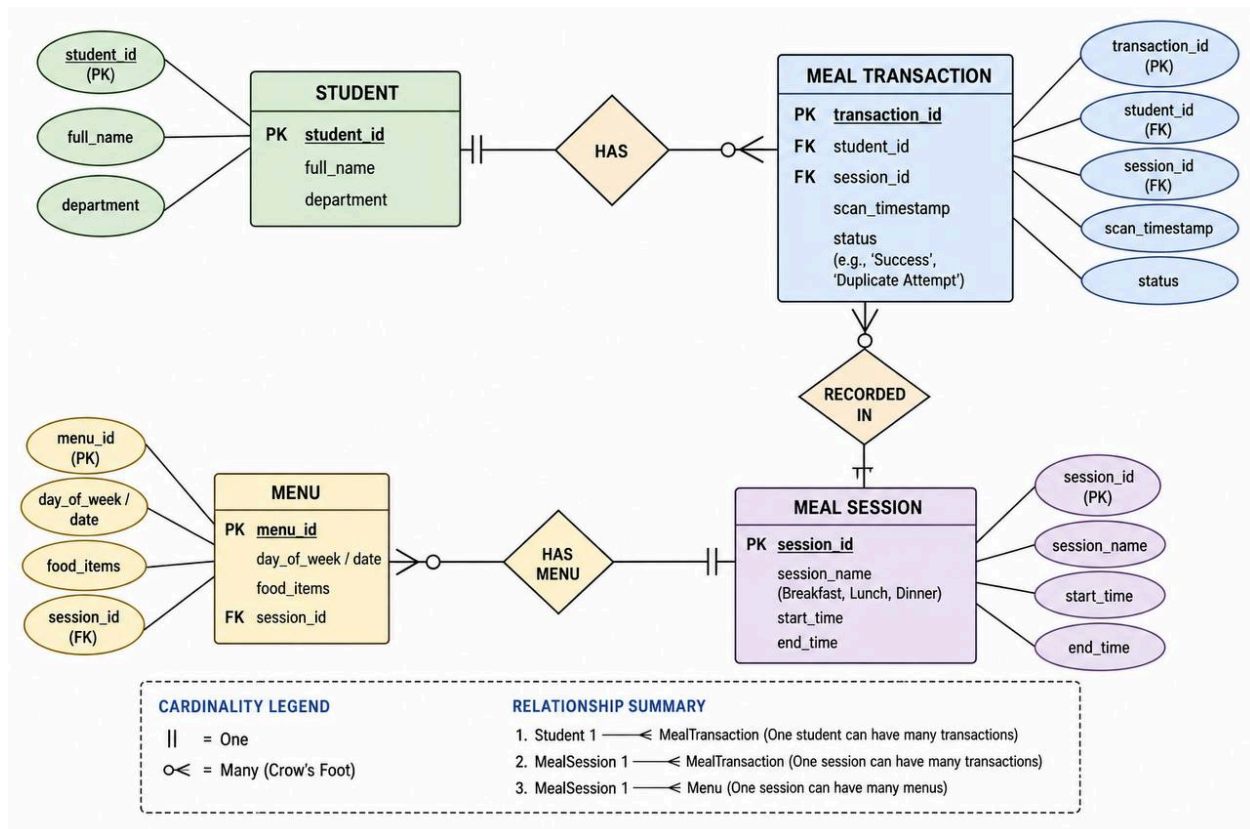
2.3 Relationship Analysis

The relationships between the entities are:

- One student can have many meal transactions
- One meal session can have many meal transactions
- One meal session can contain multiple menu records

These relationships help maintain data integrity and support efficient data retrieval.

2.4 ER Diagram



2.5 Logical Schema

STUDENT

- student_id (Primary Key)
- student_name
- department
- year

MEAL_TRANSACTION

- transaction_id (Primary Key)
- student_id (Foreign Key)
- session_id (Foreign Key)
- transaction_time
- status

MEAL_SESSION

- session_id (Primary Key)
- session_name
- session_time

MENU

- menu_id (Primary Key)
- session_id (Foreign Key)
- food_name
- menu_date

2.6 Normalization

The database tables are normalized to reduce redundancy and improve data consistency.

First Normal Form (1NF)

All attributes contain atomic values, and repeating groups are removed.

Second Normal Form (2NF)

Partial dependencies are removed, and all non-key attributes fully depend on the primary key.

Third Normal Form (3NF)

Transitive dependencies are removed to improve data integrity.

Boyce-Codd Normal Form (BCNF)

All determinants are candidate keys, ensuring better normalization.

- The database tables were normalized through different normalization stages to ensure efficient data organization and integrity. The figure below presents the normalization structure used in the system.

Database Normalization – Cafeteria System

Phase 1: Unnormalized Form (0NF)

All data is stored in one large table:

Student ID	Name	Department	Date	Meal Session	Menu Details	Time
101	Abel	CS	2026-05-05	Breakfast	Bread,tea,rice	8:00 am

- Data redundancy (same student info repeated many times)
- Wastes storage
- Leads to inconsistency

Phase 2: First Normal Form (1NF)

Rule: Each field contains only atomic (single) values.

Application:

- Remove multi-valued attributes (e.g., "Rice, Stew, Salad")
- Each column holds a single value

Result:

- Tables have primary keys
- Data is structured properly

Student ID	Name	Department	Date	Meal Session	Menu Item	Time
101	Abel	CS	2026-05-05	Breakfast	Bread	8:00 am
101	Abel	CS	2026-05-05	Breakfast	tea	8:00 am
101	Abel	CS	2026-05-05	Breakfast	rice	8:00 am

Phase 3: Second Normal Form (2NF)

Composite key:

(StudentID, Date, MealSession)

Non-key attributes:

- StudentName
- Department

Problem:

- StudentName and Department depend only on StudentID, not the whole key.

Rule:

- Must be in 1NF
- No partial dependency (non-key attributes depend on whole primary key)

Application:

- StudentName and Department depend only on StudentID

Fix:

- Create separate STUDENT table

Result:

- Reduced redundancy
- Easier updates

STUDENT TABLE

studentID	studentName	Department
-----------	-------------	------------

TRANSACTION Table

TransactionID	StudentID	Date	MealSession
---------------	-----------	------	-------------

Phase 4: Third Normal Form (3NF)

Rule:

- Must be in 2NF
- No transitive dependency (non-key depends on non-key)

MEAL_SESSION Table

SessionID	SessionName	StartTime
-----------	-------------	-----------

MENU Table

MenuID	MenuName
--------	----------

Phase 5: BCNF (Boyce-Codd Normal Form)

Rule:

- Every determinant must be a candidate key

Application:

- SessionID determines StartTime

Example:

If: SessionID $\rightarrow$ StartTime and SessionID is a candidate key, then the table satisfies BCNF.

■

Result:

- All tables are fully normalized
- No redundancy remains

Summary:

- 1NF: Atomic values and primary keys
- 2NF: Removed partial dependencies
- 3NF: Removed transitive dependencies
- BCNF: All determinants are candidate keys

2.7 Input and Output Design

Inputs

- Student registration form
- Meal transaction form
- Meal session form
- Menu information form

Outputs

- Meal transaction reports
- Student meal history reports
- Meal usage summaries

CHAPTER THREE

IMPLEMENTATION

3.1 MySQL Implementation

The relational database is implemented using MySQL.

The implementation includes:

- Table creation
- Constraints
- Relationships
- Queries
- Report generation

Example: This SQL commands create and select the cafeteria management database used in the system.

```
CREATE DATABASE IF NOT EXISTS cafeteria_system;
```

```
USE cafeteria_system;
```

```
MariaDB [cafeteria_system]> DESCRIBE meal_session;
```

Field	Type	Null	Key	Default	Extra
session_id	int(11)	NO	PRI	NULL	auto_increment
session_name	enum('Breakfast','Lunch','Dinner')	NO		NULL	
start_time	time	NO		NULL	
end_time	time	NO		NULL	

```
4 rows in set (0.006 sec)
```

```
MariaDB [cafeteria_system]>
```

```
MariaDB [cafeteria_system]> DESCRIBE meal_transaction;
```

Field	Type	Null	Key	Default	Extra
transaction_id	int(11)	NO	PRI	NULL	auto_increment
student_id	int(11)	NO	MUL	NULL	
session_id	int(11)	NO	MUL	NULL	
transaction_date	date	NO		NULL	
scan_timestamp	datetime	NO		current_timestamp()	
status	enum('Success','Duplicate Attempt')	NO		Success	

```
6 rows in set (0.007 sec)
```

```
MariaDB [cafeteria_system]>
```

Example: The query retrieves the total number of meal transactions performed each day by grouping records according to transaction dates.

Query 1 — Daily Meal Transactions

```
SELECT transaction_date,  
       COUNT(*) AS total_transactions  
FROM meal_transaction  
GROUP BY transaction_date  
ORDER BY transaction_date;
```

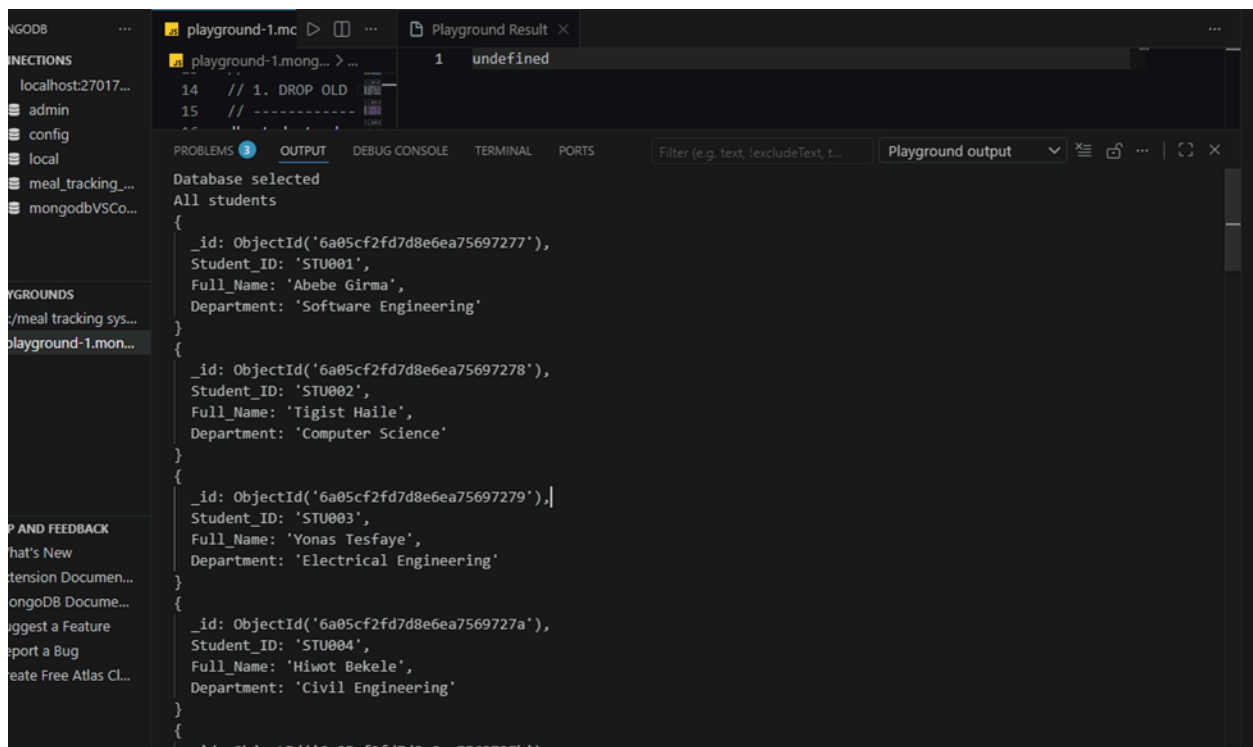
3.2 MongoDB Implementation

The non-relational database is implemented using MongoDB.

Collections are created to store:

- Student records
- Meal transaction records
- Meal session information
- Menu information

The following screenshot shows the student records stored in the MongoDB database. The database contains student information such as student ID, full name, and department.



The screenshot displays a MongoDB playground interface. The left sidebar shows a list of collections: 'admin', 'config', 'local', 'meal_tracking...', and 'mongodbVSCo...'. The main area shows the 'Playground Result' tab with a single document selected. The document is a JSON object representing a student record. The output pane shows the following JSON data:

```
{
  "_id": ObjectId('6a05cf2fd7d8e6ea75697277'),
  "Student_ID": 'STU001',
  "Full_Name": 'Abebe Girma',
  "Department": 'Software Engineering'
}
```

```
{
  "_id": ObjectId('6a05cf2fd7d8e6ea75697278'),
  "Student_ID": 'STU002',
  "Full_Name": 'Tigist Haile',
  "Department": 'Computer Science'
}
```

```
{
  "_id": ObjectId('6a05cf2fd7d8e6ea75697279'),
  "Student_ID": 'STU003',
  "Full_Name": 'Yonas Tesfaye',
  "Department": 'Electrical Engineering'
}
```

```
{
  "_id": ObjectId('6a05cf2fd7d8e6ea7569727a'),
  "Student_ID": 'STU004',
  "Full_Name": 'Hiwot Bekele',
  "Department": 'Civil Engineering'
}
```


Example: This MongoDB query retrieves transaction records and inserts a new meal transaction into the database.

```
use cafeteria_system

db.transactions.find()

db.transactions.insertOne({
  student_name: "Elyas Demamu",
  session_name: "Breakfast",
  status: "Success" })
```

3.3 Sample Data Insertion

Sample data is inserted into the database to test functionality and verify relationships.

Examples include:

- Student records
- Meal session records
- Menu records
- Meal transaction records

3.4 Queries and Reports

The system supports several queries and reports such as:

- Total number of meal transactions
- Student meal history
- Meal usage reports
- Session-based transaction reports

```

MariaDB [cafeteria_system]>
MariaDB [cafeteria_system]> SELECT
  -> mt.transaction_date,
  -> ms.session_name,
  -> COUNT(*) AS total
  -> FROM meal_transaction mt
  -> JOIN meal_session ms ON ms.session_id = mt.session_id
  -> GROUP BY mt.transaction_date, ms.session_name
  -> ORDER BY mt.transaction_date, ms.start_time;
+-----+-----+-----+
| transaction_date | session_name | total |
+-----+-----+-----+
| 2026-05-12      | Breakfast    | 2     |
| 2026-05-12      | Lunch        | 2     |
| 2026-05-12      | Dinner       | 1     |
| 2026-05-13      | Lunch        | 1     |
+-----+-----+-----+
4 rows in set (0.001 sec)

MariaDB [cafeteria_system]>

```

```

MariaDB [cafeteria_system]>
MariaDB [cafeteria_system]> SELECT
  -> s.student_id,
  -> s.full_name,
  -> COUNT(*) AS meals_taken
  -> FROM meal_transaction mt
  -> JOIN student s ON s.student_id = mt.student_id
  -> GROUP BY s.student_id, s.full_name
  -> ORDER BY meals_taken DESC, s.full_name
  -> LIMIT 10;
+-----+-----+-----+
| student_id | full_name      | meals_taken |
+-----+-----+-----+
| 1          | Elroy Melaku   | 2           |
| 3          | Elisabeth Alemneh | 1           |
| 2          | Elshaday Alem  | 1           |
| 4          | Elyas Demamu   | 1           |
| 5          | Henok Tadesse  | 1           |
+-----+-----+-----+
5 rows in set (0.006 sec)

MariaDB [cafeteria_system]>

```

3.5 Testing and Validation

Testing is performed to ensure that:

- Data is stored correctly
- Relationships function properly
- Queries produce accurate results
- Reports are generated successfully

The system is validated using sample transaction data.

3.6 Challenges Encountered

During the development of the project, several challenges were encountered, including:

- Database relationship errors
- Query optimization issues
- Data consistency problems
- Difficulty in integrating relational and non-relational databases

These issues were resolved through testing and debugging.

3.7 Future Improvements

Future improvements may include:

- Mobile application integration
- Online meal booking system
- Biometric authentication integration
- Real-time meal monitoring dashboard
- Automated report generation

PROJECT TIMELINE

Week	Activity	Duration
Week 1	Group formation and topic selection	1 Week
Week 2	Chapter 1 – Introduction	1 Week
Week 3–4	Chapter 2 – Database Design	2 Weeks
Week 5–6	Chapter 3 – Implementation	2 Weeks
Week 7	Testing and Final Report Writing	1 Week
Week 8	Submission and Presentation	1 Week

CONCLUSION

The Meal Management System database project is designed to improve the management of university meal service operations.

The system supports organized storage of student information, meal transactions, meal sessions, and menu records.

By using both MySQL and MongoDB, the project demonstrates the implementation of relational and non-relational database concepts.

The proposed system improves efficiency, reduces redundancy, supports accurate transaction recording, and assists in report generation.

REFERENCES

[1] R. Elmasri and S. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

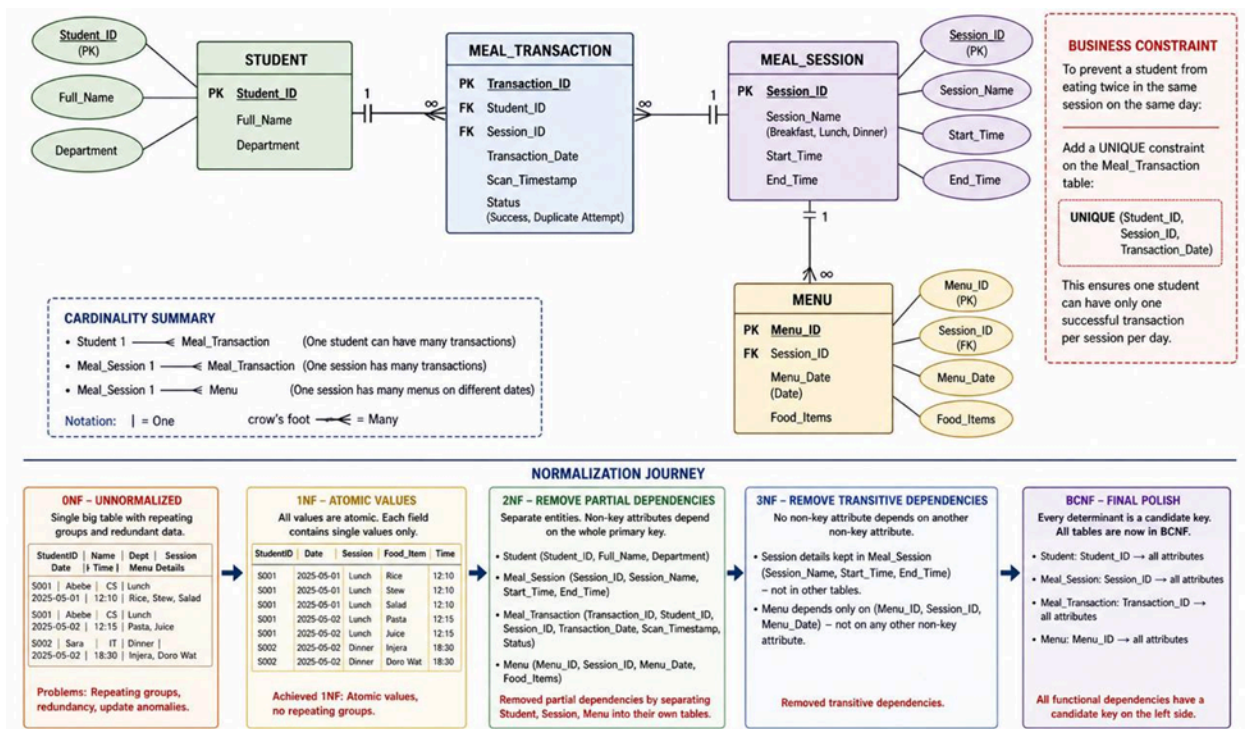
[2] C. Coronel and S. Morris, Database Systems: Design, Implementation, and Management, 13th ed. Cengage Learning, 2019.

[3] MongoDB Documentation. Available: <https://www.mongodb.com/docs/>

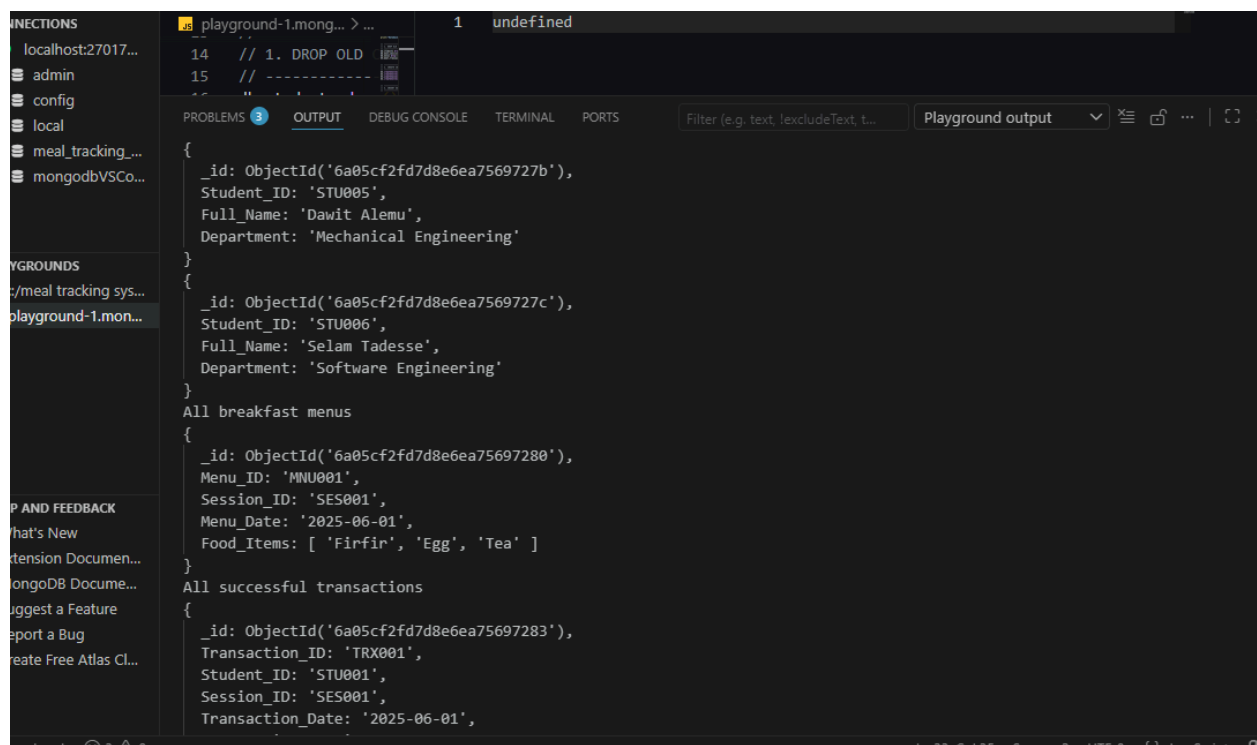
[4] MySQL Documentation. Available: <https://dev.mysql.com/doc/>

Supplementary Information

Table and Normalization Summary



MongoDB Queries



MongoDb Collection Screenshots

The screenshot displays the MongoDB Compass web interface. On the left, the 'CONNECTIONS (1)' sidebar shows a tree view of databases and collections. The 'meal_tracking_db' database is expanded, revealing several collections: 'NEW_COLLECTION_NAME', 'meal_sessions', 'meal_transactions', 'menus', and 'students'. The 'students' collection is selected and highlighted.

The main panel on the right shows the 'Find' tab with a query bar containing the text: 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', 'EXPORT DATA', and 'EXPORT CODE'. The document list shows 25 documents, with the first document selected. The selected document is displayed in a detailed view, showing its structure and values.

The selected document is as follows:

```
{
  "_id": ObjectId('6a05d5321f9ce9727c7a3121'),
  "Student_ID": "STU001",
  "Full_Name": "Abebe Girma",
  "Department": "Software Engineering"
}
```

The second document in the list is also shown:

```
{
  "_id": ObjectId('6a05d5321f9ce9727c7a3122'),
  "Student_ID": "STU002",
  "Full_Name": "Tigist Haile",
  "Department": "Computer Science"
}
```

The third document is:

```
{
  "_id": ObjectId('6a05d5321f9ce9727c7a3123'),
  "Student_ID": "STU003",
  "Full_Name": "Yonas Tesfaye",
  "Department": "Software Engineering"
}
```

The fourth document is:

```
{
  "_id": ObjectId('6a05d5321f9ce9727c7a3124'),
  "Student_ID": "STU004",
  "Full_Name": "Hiwot Bekele",
  "Department": "Civil Engineering"
}
```